Fuzzing Guiado por Contrato para APIs REST

Title: Contract-Guided Fuzzing for REST APIs

Levi da Rosa Gomes (aluno)[1], **Fábio Austrian Zanin (orientador)**[1]

[1] Universidade Regional Integrada do Alto Uruguai e das Missões
Departamento de Engenharias e Ciência da Computação
Caixa Postal 743 – 99.709-910 – Erechim – RS – Brasil

103495@uricer.edu.br, faz@uricer.edu.br

Abstract. *Divergences between REST API implementations and their OpenAPI contracts can affect integration reliability and reveal robustness, conformance, and access-control anomalies. This work presents a black-box, contract-guided fuzzer and investigates the use of a fuzzy inference system to prioritize the reported anomalies. The tool parses a supported subset of OpenAPI, generates baseline and mutated inputs, executes HTTP requests, detects contractual divergences, and preserves reproducible artifacts. Four APIs supported the development and audit of the prioritization model; after its revision was frozen, Meilisearch was used as an external holdout. The development pilot executed 958 requests and produced 161 behavioral families. The initial model reproduced the same categorical decisions as a status-only baseline, with accuracy 0.298. On 122 supported holdout families, the revised model agreed with a blind operational reference in 120 cases, compared with 4 for the baseline. These partial results concern technical priority, not actual security impact; the final experimental protocol remains subject to advisor review.*
Keywords *REST APIs, OpenAPI, Fuzzing, Contract Testing, Fuzzy Logic.*

Resumo. *Divergências entre implementações de APIs REST e seus contratos OpenAPI podem afetar a confiabilidade das integrações e revelar anomalias de robustez, conformidade e controle de acesso. Este trabalho apresenta um fuzzer de caixa-preta guiado por contrato e investiga o uso de um sistema de inferência fuzzy para priorizar as anomalias reportadas. A ferramenta interpreta um subconjunto de OpenAPI, gera entradas-base e mutações, executa requisições HTTP, detecta divergências contratuais e preserva artefatos reproduzíveis. Quatro APIs apoiaram o desenvolvimento e a auditoria do modelo de priorização; após o congelamento de sua revisão, o Meilisearch foi utilizado como holdout externo. O piloto de desenvolvimento executou 958 requisições e produziu 161 famílias comportamentais. O modelo inicial reproduziu as mesmas decisões categóricas de um baseline baseado apenas em status, com acurácia 0,298. Nas 122 famílias sustentadas do holdout, o modelo revisado concordou com uma referência operacional cega em 120 casos, contra 4 do baseline. Esses resultados parciais tratam de prioridade técnica, não de impacto real de segurança; o protocolo experimental final permanece sujeito à avaliação do orientador.*
Palavras-Chave *APIs REST, OpenAPI, Fuzzing, Teste de Contrato, Lógica Fuzzy.*

1. Introdução

APIs REST tornaram-se parte recorrente da infraestrutura de sistemas modernos, conectando serviços, aplicações *web* e dispositivos móveis [Golmohammadi et al. 2023]. Nesse cenário,

falhas de robustez e inconsistências entre o comportamento da API e o contrato publicado afetam a confiabilidade das integrações e o custo de manutenção. Testes funcionais tradicionais validam cenários esperados, mas podem deixar de exercitar valores limite, formatos inválidos e combinações pouco usuais de parâmetros.

Contratos como a especificação OpenAPI descrevem operações, estruturas, tipos, restrições e respostas esperadas. Eles podem orientar a geração sistemática de entradas válidas e inválidas e servir como oráculo parcial para identificar divergências entre especificação e implementação. Essa abordagem permite observar a API externamente, sem instrumentação de código, em um contexto *black-box* relacionado à noção de contrato de software [Meyer 1992].

Este trabalho apresenta uma ferramenta de *fuzzing* automatizado para APIs REST, implementada em Python e orientada por um subconjunto de OpenAPI. O fluxo interpreta o contrato, gera valores-base e mutações, materializa um plano de execução, envia requisições, confronta as respostas com o contrato e produz artefatos rastreáveis. Um sistema de inferência *fuzzy* Mamdani atribui prioridade técnica às anomalias reportadas depois da classificação contratual.

A questão central é: *Quais contribuições e limitações a combinação de fuzzing black-box orientado por OpenAPI, deduplicação e priorização fuzzy apresenta para a triagem de divergências em APIs REST?*

A avaliação é organizada em três dimensões complementares: (i) execução e completude das evidências de anomalias contratuais e comportamentais; (ii) estabilidade dos comportamentos entre campanhas; e (iii) redução por deduplicação e coerência da prioridade técnica para triagem.

Quatro APIs foram utilizadas como pilotos de desenvolvimento, permitindo auditar a configuração inicial e formular um modelo revisado. Após o congelamento das regras, uma quinta API foi empregada como *holdout* externo. A prioridade foi comparada com uma referência operacional cega, construída sem revelar o escore e assistida por agentes independentes. Esses resultados são evidência parcial e base para discutir o delineamento definitivo; não implicam superioridade geral sobre outros *fuzzers* nem estimativa de impacto real de segurança.

O escopo limita-se a APIs REST descritas em OpenAPI. Não são abrangidos sequenciamento geral com estado, fluxos completos de OAuth2, avaliação de carga, cobertura de código ou confirmação automática de vulnerabilidades. Um achado pode representar *contract drift*, incompatibilidade funcional, robustez ou comportamento de controle de acesso e ainda exigir interpretação antes de ser tratado como defeito.

Integração contínua constitui um contexto de uso potencial e uma motivação, mas o trabalho não implementa nem avalia integração com uma esteira.

O restante deste artigo está organizado da seguinte forma. A Seção 2 apresenta os conceitos necessários. A Seção 3 posiciona a proposta diante das ferramentas relacionadas. A Seção 4 descreve o desenho da avaliação, e a Seção 5 apresenta a ferramenta. A Seção 6 registra os resultados parciais. Por fim, a Seção 7 sintetiza o marco atual e suas limitações.

2. Fundamentação Teórica

Esta seção reúne os conceitos necessários para compreender a proposta sem antecipar o protocolo experimental. O encadeamento adotado distingue três etapas: o contrato fornece uma descrição

processável e expectativas parciais; o teste *black-box* produz observações e evidências; e a inferência *fuzzy* ordena os achados já detectados. Essa separação é importante porque gerar uma requisição, reconhecer uma divergência e atribuir prioridade são problemas relacionados, mas não equivalentes.

2.1. REST, semântica HTTP e contratos processáveis

REST é um estilo arquitetural para sistemas distribuídos, definido por restrições como identificação de recursos, interface uniforme, comunicação sem estado entre requisições e transferência de representações [Fielding 2000]. HTTP fornece a semântica observável da interação: métodos expressam a intenção da requisição, enquanto códigos de status e cabeçalhos comunicam características da resposta [Fielding et al. 2022]. Uma API usar HTTP e JSON, portanto, não basta para demonstrar que ela satisfaz todas as restrições de REST; esses elementos são apenas parte da interface que um teste externo consegue observar.

A OpenAPI Specification (OAS) descreve APIs HTTP de forma independente de linguagem. Um documento pode declarar caminhos, operações, parâmetros, corpos, respostas e esquemas de segurança [OpenAPI Initiative 2021]. Em OpenAPI 3.1, a descrição dos dados se alinha ao JSON Schema, cujo vocabulário permite formular asserções estruturais com palavras-chave como `type`, `required`, `enum`, limites numéricos, padrões e propriedades [JSON Schema 2022]. Essas declarações tornam uma parte do acordo entre cliente e serviço legível por ferramentas.

Há uma analogia útil com *Design by Contract*: precondições expressam obrigações do cliente, pós-condições expressam garantias do fornecedor e invariantes preservam propriedades do objeto [Meyer 1992]. A analogia não estabelece equivalência formal. OpenAPI não expressa necessariamente invariantes, dependências entre chamadas ou todas as regras de negócio; além disso, o documento publicado pode estar incompleto ou desatualizado.

Neste trabalho, o contrato é uma descrição processável do comportamento esperado e não uma especificação completa da aplicação. O *fuzzer* interpreta um subconjunto de OpenAPI para construir requisições e conferir propriedades observáveis. Logo, toda conclusão deve ser delimitada tanto pelo que o contrato declara quanto pelo que a ferramenta efetivamente suporta.

2.2. *Fuzzing black-box*, oráculos e evidência

Fuzzing automatiza a produção e a execução de muitas entradas [Golmohammadi et al. 2023]. Uma estratégia geracional constrói casos a partir de um modelo, enquanto uma estratégia mutacional transforma casos existentes. Em APIs REST, o contrato permite preservar a estrutura de parâmetros e corpos ao gerar casos potencialmente válidos e também orientar teste negativo, no qual obrigatoriedade, tipo, faixa ou outra restrição é deliberadamente violada. RESTest, por exemplo, distingue casos nominais de casos defeituosos e explicita oráculos diferentes para requisições que deveriam ser aceitas ou rejeitadas [Martin-Lopez et al. 2020]; RestTestGen concretiza a mesma separação em estratégias nominal e de erro, com mutadores próprios [Corradini et al. 2022].

No modo *black-box*, a geração e a decisão não recebem cobertura de código nem instrumentação interna. Contrato, configuração, requisições e respostas são as fontes de evidência disponíveis. Isso amplia a aplicabilidade a serviços cujo código não está

acessível, mas limita a capacidade de explicar causas internas. Autenticação, estado persistente, dados realistas e dependências entre operações continuam sendo obstáculos relevantes [Golmohammadi et al. 2023]. RESTler mostra uma resposta específica a parte desse problema: infere relações produtor–consumidor do contrato, preserva valores obtidos nas respostas e explora sequências de requisições com realimentação da execução [Atlidakis et al. 2019]. Essa exploração com estado é diferente de apenas repetir operações isoladas.

Um oráculo de teste decide se a observação satisfaz alguma expectativa. Status de servidor, status não declarado, conformidade do corpo com o esquema e aceitação de uma entrada inválida são exemplos de propriedades verificáveis a partir da interface. Nenhuma delas, isoladamente, comprova que toda a operação está correta. O contrato atua, portanto, como *oráculo parcial*: permite detectar contradições relativas às declarações disponíveis, mas não cobre efeitos de negócio, intenção do fornecedor, impacto ou segurança. Estudos da área mostram que ferramentas baseadas em OpenAPI adotam combinações distintas de geração e oráculos, e que a incompletude dos esquemas limita a exploração *black-box* [Golmohammadi et al. 2023, Zhang e Arcuri 2023].

Uma observação só é útil para análise posterior se permanecer ligada às condições que a produziram. RESTler associa o erro à sequência de requisições e agrupa ocorrências provavelmente redundantes; Schemathesis combina redução do caso que falha com deduplicação de erros; e RestTestGen pode registrar em JSON as requisições, respostas e avaliações dos oráculos [Atlidakis et al. 2019, Hatfield-Dodds e Dygalo 2021, Corradini et al. 2022]. Esses mecanismos não são idênticos, mas evidenciam três necessidades distintas: *rastreabilidade* relaciona a observação à sua origem; *reprodução* preserva informação suficiente para provocar novamente o comportamento; e *deduplicação* reduz ocorrências equivalentes para a triagem. Um grupo deduplicado não corresponde automaticamente a uma causa raiz.

O projeto adota esse vocabulário de forma operacional. Uma ocorrência é o resultado individual de um caso executado. Uma divergência é uma propriedade observada que contradiz uma expectativa interpretada do contrato ou de um detector. Um achado é a evidência estruturada dessa divergência, vinculada ao caso e à resposta. Assinaturas e famílias agrupam padrões observáveis para evitar que repetições dominem a análise. Em um possível ciclo de integração contínua, um resultado útil para triagem deve ser produzido sob orçamento conhecido, manter essa cadeia de evidências, separar campanha incompleta de comportamento do alvo e apresentar volume tratável. Esses são critérios de uso adotados pelo estudo; o trabalho não implementa nem avalia uma esteira de CI.

2.3. Inferência *fuzzy* e prioridade técnica

Em um conjunto clássico, a pertinência é binária. Em um conjunto *fuzzy*, uma função associa cada elemento a um grau no intervalo entre zero e um [Zadeh 1965]. Esse grau representa o quanto a observação pertence a um conceito gradual definido pelo modelo; não deve ser interpretado como probabilidade. Termos linguísticos como baixa, média e alta podem, assim, representar regiões sobrepostas de uma variável.

O sistema implementado organiza a inferência em quatro passos conceituais: calcula os graus de pertinência, avalia regras linguísticas, agrega as conclusões ativadas e desfuzzifica o conjunto resultante em um valor numérico. Ele adota a abordagem linguística de Mamdani, cujas regras têm a forma “se ... então ...” e mantêm consequentes *fuzzy* [Mamdani e Assilian 1975]. As

funções de pertinência, regras, operadores e o método de desfuzzificação são escolhas do projeto; a fonte histórica não determina os parâmetros adequados para este domínio.

Prioridade também exige delimitação. Na literatura de regressão, priorizar casos de teste significa alterar sua ordem para atingir antes um objetivo, como revelar falhas [Elbaum et al. 2002]; abordagens *fuzzy* já foram empregadas para ordenar esses casos antes da execução [Alakeel 2014]. O problema deste projeto ocorre depois da execução: os analisadores já produziram achados, e o sistema Mamdani ordena quais evidências merecem inspeção primeiro.

Por isso, *prioridade técnica* designa neste TCC a urgência relativa de triagem segundo os sinais presentes nos artefatos. Ela não equivale a severidade universal, impacto de negócio, explorabilidade ou probabilidade de vulnerabilidade. Uma vulnerabilidade requer que condições e consequências de segurança sejam confirmadas; uma divergência contratual pode ser relevante para compatibilidade ou robustez sem constituir vulnerabilidade. O FIS não detecta a divergência nem substitui essa análise: ele recebe achados já classificados e produz uma ordenação interpretável. Suas entradas, funções e regras são escolhas do projeto e precisam ser descritas e avaliadas empiricamente, em vez de serem tratadas como consequência automática da literatura *fuzzy*.

3. Trabalhos Relacionados

Esta seção compara estratégias de teste automatizado de APIs REST que usam um contrato como entrada. A análise é exclusivamente bibliográfica: nenhuma dessas ferramentas foi executada como linha de base experimental. O objetivo não é escolher uma vencedora, mas identificar como diferentes trabalhos tratam geração, estado, oráculos e apresentação dos resultados, preservando os limites das versões e dos experimentos descritos nas fontes.

Foram selecionados Schemathesis, RESTler, RESTest e RestTestGen porque possuem fontes primárias acessíveis e cobrem decisões diretamente relacionadas ao projeto. A comparação observa: formato de contrato e acesso *black-box* ou *white-box*; geração válida, inválida e com estado; oráculos e classes de falha; artefatos, reprodução e deduplicação; priorização de resultados; e relação declarada com integração contínua.

3.1. Estratégias de geração e tratamento de estado

As quatro abordagens partem de uma descrição de API, mas não constroem o espaço de testes da mesma forma. RESTler compila Swagger/OpenAPI em uma gramática, infere dependências produtor–consumidor e usa valores extraídos das respostas para formar sequências com estado. A execução fornece realimentação para descartar sequências inviáveis e aprofundar estados alcançáveis [Atlidakis et al. 2019]. Seu ponto central é a ordem entre operações e a propagação de recursos dinâmicos.

RESTest trabalha no nível de uma operação por caso. Ele gera entradas nominais e variantes inválidas e pode enriquecer OpenAPI com IDL4OAS para expressar dependências entre parâmetros que o contrato comum não contém. A geração baseada em restrições tenta satisfazer ou violar deliberadamente essas dependências [Martin-Lopez et al. 2020]. Assim, sua noção de dependência está principalmente dentro da requisição, enquanto a exploração com estado entre operações não é o foco do artigo-base.

Schemathesis deriva geradores e propriedades de OpenAPI ou GraphQL por meio de teste baseado em propriedades. A fonte descreve geração de entradas válidas e negativas, redução

automática de casos que falham e testes em máquina de estados, estes dependentes de relações entre operações como os `links` de OpenAPI [Hatfield-Dodds e Dygalo 2021]. RestTestGen, por sua vez, oferece componentes extensíveis para análise, modelos de dados, provedores de valores, mutadores, ordenação, execução e oráculos. Sua estratégia nominal ordena operações a partir de um grafo de dependências e reutiliza valores observados; a estratégia de erro aplica mutações a interações nominais bem-sucedidas [Corradini et al. 2022].

Essas diferenças produzem custos e alcances distintos. Na comparação histórica de Corradini et al., RestTestGen, RESTler, bBOXRT e RESTest foram avaliados em 14 serviços com versões e configurações de 2021; Schemathesis não integrou esse corpus. Robustez da ferramenta, enumeração de sequências e diversidade de dados competiram pelo mesmo orçamento de teste [Corradini et al. 2021]. Os resultados ajudam a explicar os compromissos, mas não estabelecem superioridade atual ou universal de uma estratégia.

3.2. Oráculos, evidências e triagem

As fontes também divergem no que consideram um resultado relevante. O artigo inicial do RESTler trata respostas 5xx como falhas observáveis e associa cada uma à sequência que a produziu. Como ocorrências repetidas são frequentes, propõe *bucketization* por sufixos de sequências para facilitar a inspeção; os autores relatam que as falhas do estudo puderam ser reproduzidas [Atlidakis et al. 2019]. Esse agrupamento é específico ao modelo do RESTler e não prova causa raiz comum.

RESTest explicita cinco oráculos: ausência de 5xx, conformidade da resposta com OAS, rejeição de violações de parâmetros, rejeição de violações de dependências e não rejeição de requisições consideradas válidas. Também gera testes Java executáveis e oferece execução *offline* ou *online*; a fonte apresenta esta última como aplicável a um ambiente de integração contínua [Martin-Lopez et al. 2020]. O artigo não descreve uma ordenação pós-execução ou uma deduplicação geral dos achados.

Schemathesis deriva propriedades de semânticas HTTP e do esquema, incluindo status não declarado, corpo incompatível e rejeição de entrada inválida. O mecanismo subjacente de teste baseado em propriedades fornece redução e deduplicação de erros, e os testes derivados podem integrar uma suíte existente ou ser executados diretamente [Hatfield-Dodds e Dygalo 2021]. RestTestGen separa oráculos de status e esquema, registra sequências, requisições, respostas e resultados dos oráculos em relatórios JSON e pode materializar casos com REST-assured [Corradini et al. 2022]. Na fonte-base, nenhuma das duas abordagens apresenta um classificador geral de prioridade técnica dos achados.

Para tornar os limites verificáveis, a Tabela 1 resume apenas capacidades documentadas nas fontes citadas. “Não demonstrado” significa que o aspecto não foi caracterizado no trabalho consultado, e não que seja impossível ou esteja ausente de toda versão posterior.

3.3. Posicionamento da proposta

A comparação evidencia que geração por OpenAPI, casos negativos, validação de status ou esquema, sequências com estado e redução de resultados já aparecem em trabalhos anteriores. Também mostra que “priorização” pode se referir a coisas diferentes: escolher operações e valores durante a geração, guiar a busca por sequências ou ordenar evidências depois que a execução terminou. O projeto atua no terceiro sentido.

Tabela 1. Comparação bibliográfica das estratégias selecionadas, conforme as fontes citadas no texto

Ferramenta	Contrato, acesso e geração	Estado e casos negativos	Oráculos e evidências	Priorização e CI
Schemathesis	OpenAPI/GraphQL; *black-box* ou *white-box* em processo; geração baseada em propriedades	Entradas válidas e negativas; máquina de estados dependente das relações disponíveis	Propriedades de HTTP/esquema; redução automática e deduplicação de erros no mecanismo subjacente	Sem ordenação pós-execução demonstrada; integrável a suítes de teste
RESTler	Swagger/OpenAPI; *black-box*; gramática e dicionários	Sequências com estado, relações produtor–consumidor e valores mutados	5xx no oráculo-base; sequência preservada e *bucketization* de ocorrências	Busca prioriza exploração, não achados; CI não caracterizada na fonte
RESTest	OpenAPI com IDL4OAS opcional; *black-box*; geração aleatória ou por restrições	Uma operação por caso; entradas nominais e inválidas, inclusive dependências entre parâmetros	Cinco oráculos; testes executáveis; deduplicação não demonstrada	Sem ordenação de achados; execução *online* apresentada para CI
RestTestGen	OpenAPI; *black-box*; provedores de valores e mutadores extensíveis	Grafo de dependências, reuso de valores e estratégias nominal/de erro	Oráculos de status e esquema; relatórios JSON e testes REST-assured	Prioriza geração/ordem, não achados; CI não avaliada na fonte
Projeto deste TCC	Subconjunto de OpenAPI; *black-box*; caso-base e mutações tipadas	Níveis L0, L1 e L2; dependências com estado apenas parciais	Detectores contratuais; plano, respostas, manifestos e famílias deduplicadas	FIS Mamdani pós-execução; CI apenas como contexto de uso

A lacuna investigada não é a ausência de *fuzzers* de API nem a superioridade de uma técnica de geração. O estudo examina as contribuições e limitações de uma combinação específica: *fuzzing black-box* orientado por OpenAPI, evidências rastreáveis, deduplicação por comportamento observável e prioridade *fuzzy* pós-execução. Essa formulação é restrita ao corpus bibliográfico consultado e será avaliada nas aplicações do estudo. Como não há comparação experimental com as ferramentas desta seção, os resultados não autorizam afirmar maior cobertura, eficácia ou qualidade do projeto em relação a elas.

4. Metodologia

Para responder à questão de pesquisa, adota-se uma avaliação aplicada e experimental, baseada em execuções locais controladas. Esta seção define o papel das aplicações, a geração dos casos, as condições de repetição e as métricas usadas para analisar os artefatos produzidos. O protocolo distingue as evidências históricas de desenvolvimento da coleta final planejada.

4.1. Desenho do Estudo e Papel das Aplicações

VAmPI, Spring Petclinic REST, crAPI e RealWorld formaram o conjunto usado para desenvolver e caracterizar a ferramenta. Os resultados dessas aplicações permitiram auditar a configuração inicial do mecanismo de priorização e formular sua revisão. O Meilisearch v1.45.1 foi executado somente depois dessa revisão, como uma aplicação externa de confirmação, sem participar do ajuste das novas regras.

As aplicações foram executadas localmente em versões fixadas e reiniciáveis. Contratos, imagens, configuração da ferramenta e procedimentos de inicialização foram registrados para separar mudanças do objeto avaliado de variações do ambiente.

A coleta final terá cinco campanhas independentes para cada aplicação. Antes de cada campanha, a aplicação e os dados necessários serão reiniciados para que o resultado não dependa do estado deixado pela execução anterior. As quatro APIs usadas no desenvolvimento continuarão identificadas como conjunto de pilotos. O Meilisearch será analisado no mesmo conjunto de campanhas, mas manterá o papel separado de aplicação externa ao desenvolvimento do FIS.

O Meilisearch já possui cinco campanhas completas de confirmação do FIS. Elas permanecem como evidência histórica. A coleta final será formada por 25 campanhas novas, cinco por aplicação, executadas a partir da mesma revisão do repositório e com árvore de trabalho limpa. Os manifestos registrarão a revisão, os resumos criptográficos dos contratos e a configuração. Verificações prévias dos executores não integram esse conjunto final.

A repetição não existe para fazer a ferramenta sortear casos diferentes. Com o mesmo contrato, revisão e configuração, o plano de requisições tende a ser o mesmo. Ela verifica se as respostas, os achados e as assinaturas se mantêm depois do reinício. Essa distinção é necessária porque a execução ainda pode variar: alguns executores geram credenciais automaticamente, e o estado da aplicação, o tempo de resposta ou uma falha transitória podem mudar entre campanhas.

4.2. Geração dos Testes e Níveis de Mutação

A geração começa com um caso compatível com as informações interpretadas do contrato. A partir dele, o motor aplica operadores tipados aos corpos JSON. Os parâmetros de caminho, consulta e cabeçalho recebem valores gerados, mas não participam desse motor de mutações. Três níveis organizam a exploração:

- **Nível 0 (L0) — caso-base:** executa a entrada gerada para satisfazer o contrato e estabelece a referência nominal da operação;
- **Nível 1 (L1) — mutação isolada:** aplica uma alteração por caso, permitindo relacionar a resposta com um operador específico;
- **Nível 2 (L2) — mutações combinadas:** combina duas alterações compatíveis para explorar interações, preservando orçamento próprio e os casos isolados de L1.

Essa progressão distingue comportamento nominal, efeito individual e interação entre mutações. O plano também inclui, quando habilitados, testes de operações protegidas sem credencial ou com credencial inválida. Limites de casos, concorrência, atraso e tempo limite são registrados como parte da configuração.

4.3. Protocolo Experimental e Estratégia de Avaliação

Cada campanha segue o fluxo contrato, plano, execução, análise e pós-processamento. Um manifesto registra identificador do experimento, API, repetição, revisão do código, estado Git, resumos criptográficos, ambiente, configuração, horários e contagens. O plano e os resultados são correlacionados por `case_id`; campanhas incompletas ou inválidas devem permanecer identificáveis, em vez de desaparecer das evidências.

As campanhas definitivas usam a configuração já exercitada nos pilotos. O executor percorre todas as operações suportadas pelo módulo que interpreta o contrato e executa os níveis

L0, L1 e L2. O limite configurado é de 20 casos de mutação isolada por corpo no L1 e de 10 pares compatíveis no L2. Esses limites são fixos; a quantidade efetiva pode ser menor quando o contrato oferece menos parâmetros, esquemas ou combinações aplicáveis. A concorrência fica em 1, o intervalo entre requisições em 250 ms e o tempo limite em 10 s. A autenticação automática é usada somente nos alvos em que o executor consegue configurá-la de forma reproduzível.

Aqui, contrato significa o arquivo OpenAPI de cada alvo. Ele descreve as operações, caminhos, parâmetros, corpos e respostas que o executor consegue interpretar. Por isso, contratos diferentes geram quantidades diferentes de casos e de requisições; não se espera que todas as APIs produzam o mesmo total. Cada manifesto registra o número planejado e o número executado, além da configuração usada, para que a comparação aplique a mesma regra a contratos de tamanhos distintos.

Antes da execução, a verificação de disponibilidade do alvo precisa passar. Durante a campanha, "acessível" significa que o processo continua disponível o suficiente para o executor concluir o plano. Uma resposta 4xx ou 5xx de uma operação é uma observação do teste e não invalida, por si só, a campanha. Já falha de inicialização, falha na reinicialização, encerramento do processo ou interrupção que deixe requisições sem resultado impedem a interpretação completa da execução.

Uma campanha válida tem o plano completo, um resultado relacionável a cada `case_id`, artefatos legíveis e registro da versão do alvo, da revisão do ferramenta e da configuração. Quando isso não ocorre, a campanha recebe estado inválido e a justificativa fica junto dos seus registros e artefatos; ela não entra no agregado válido e não é apagada. Se a causa puder ser corrigida, uma nova campanha é executada com o mesmo protocolo. A execução original continua preservada como evidência da falha, e a nova execução recebe seu próprio identificador.

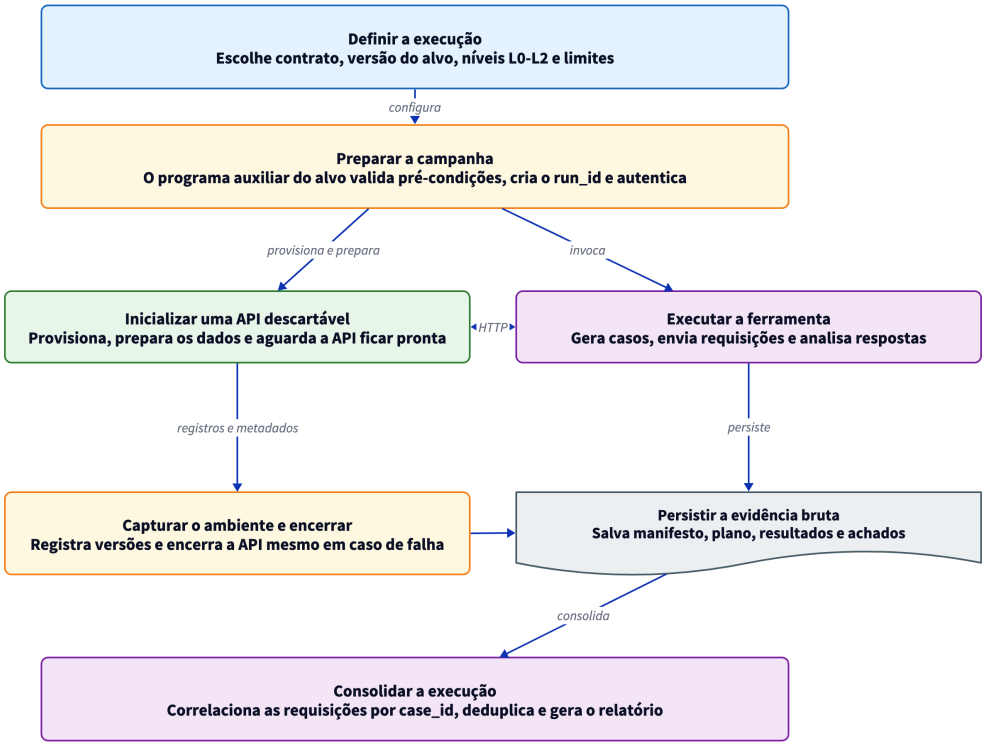


Figura 1. Ciclo de uma execução experimental isolada. O programa auxiliar específico prepara o alvo descartável e invoca a ferramenta. A ligação HTTP representa a troca de requisições e respostas; a evidência é completada com a proveniência do alvo antes da deduplicação e consolidação.

Para acompanhar uma execução sem misturar coisas diferentes, o estudo separa cinco unidades:

- uma **requisição**, que é um caso enviado à API;
- uma **observação classificada**, emitida pelo analisador para uma requisição; rejeições esperadas entram nas contagens de execução, enquanto as demais observações candidatas são os achados para triagem;
- uma **assinatura**, isto é, um identificador que agrupa observações equivalentes por aplicação, operação, classificação, status e resposta normalizada;
- uma **campanha**, que é uma execução completa contra uma API;
- uma **família de comportamento**, que reúne assinaturas que descrevem o mesmo comportamento factual por operação e tipo de divergência.

Essa separação evita contar várias mutações que produziram a mesma resposta como falhas diferentes. Ela também não afirma que cada grupo corresponde a uma causa raiz no código da aplicação.

Na avaliação histórica de desenvolvimento e confirmação do FIS, foi construída uma referência independente do seu resultado. Cada família foi apresentada com o contrato, a requisição, a resposta e a repetição observada, mas sem mostrar a classificação do analisador nem a prioridade calculada pelo FIS. Três avaliações independentes aplicaram a mesma rubrica de `LOW`,

`MEDIUM` e `HIGH`. Quando não houve maioria, uma quarta avaliação decidiu o caso sem acesso às avaliações anteriores ou ao FIS. Essa referência serve para a comparação operacional; não é uma avaliação de especialistas humanos e não confirma vulnerabilidades.

Nesse conjunto histórico, o FIS é comparado com uma regra mais simples, que usa apenas o desvio do status HTTP. As duas abordagens recebem as mesmas famílias como entrada. A análise registra a matriz de confusão, a acurácia, o macro-F1 e o erro ordinal médio, codificando `LOW`, `MEDIUM` e `HIGH` como 0, 1 e 2. O erro ordinal é a média das diferenças absolutas entre a faixa prevista e a faixa de referência. Quando uma classe estiver ausente na referência, serão apresentados separadamente o macro-F1 das três classes e o das classes observadas, declarando a ausência. Famílias sem prioridade sustentada são contabilizadas separadamente e não entram nessas métricas. Cada família contribui uma única vez para a comparação.

A coleta final não inclui nova rotulagem cega nem transferência dos rótulos históricos para os novos achados. Ela avalia execução e completude dos artefatos, divergências observadas, redução por deduplicação e estabilidade, além de descrever a distribuição dos escores e das faixas de prioridade. Sem uma referência para esses novos resultados, não serão calculadas métricas de acerto da prioridade nem inferida sua correção a partir da distribuição. O FIS permanece sem recalibração. Assim, a comparação histórica de prioridade e a coleta final constituem evidências complementares, com conjuntos e finalidades distintos.

Na coleta final, a estabilidade é medida pela presença de uma mesma assinatura nas cinco campanhas, sem uma reexecução adicional de cada requisição. Uma assinatura é considerada estável quando aparece em pelo menos quatro campanhas sob a mesma configuração. A proporção estável usa como denominador a união das assinaturas candidatas das cinco campanhas da mesma aplicação. Se não houver assinaturas, a proporção é registrada como não aplicável e o resultado zero é preservado.

4.4. Limitações e Ameaças à Interpretação

As limitações abaixo explicam o que pode influenciar os resultados e até onde eles podem ser generalizados.

O que é medido. O contrato OpenAPI funciona como uma referência parcial: ele descreve o comportamento esperado, mas pode omitir regras existentes na implementação. O ferramenta também interpreta apenas o subconjunto de OpenAPI que suporta. Por isso, uma divergência encontrada é um comportamento que merece inspeção, não uma prova automática de defeito ou vulnerabilidade. O FIS ordena a prioridade técnica desse comportamento; sua pontuação não estima impacto real, explorabilidade ou gravidade de segurança. A assinatura agrupa respostas equivalentes, mas não identifica a causa no código.

Condições da execução. Estado residual, ordem das requisições, latência da máquina e falhas internas da aplicação podem mudar o resultado de uma campanha. A reinicialização entre campanhas, concorrência 1, atraso fixo e manifestos reduzem essas diferenças. Ainda assim, não é possível garantir que toda variação do ambiente tenha sido eliminada.

Alcance das aplicações. O estudo usa cinco aplicações: quatro participaram do desenvolvimento e dos pilotos, e o Meilisearch foi mantido como aplicação externa para confirmar a priorização. Esse desenho permite testar o modelo fora do conjunto de desenvolvimento que o originou, mas não representa a variedade de todas as APIs REST. As conclusões ficam limitadas às aplicações, contratos, versões e configurações avaliados.

Força das conclusões. O número de famílias e a distribuição entre `LOW`, `MEDIUM` e `HIGH` limitam algumas métricas. A referência de prioridade foi produzida às cegas segundo uma rubrica, com avaliações independentes, mas não por especialistas humanos. O trabalho também não mede cobertura de código, não implementa integração com CI ou critério de bloqueio de qualidade e não confirma vulnerabilidades automaticamente. Essas questões ficam fora do que os resultados podem afirmar.

5. Desenvolvimento da Ferramenta

O OpenAPI Contract Fuzzer foi implementado em Python como um fluxo modular. A organização separa interpretação do contrato, geração dos casos, execução HTTP, análise das respostas, priorização e persistência dos artefatos. Esta seção descreve as responsabilidades necessárias para compreender e reproduzir o fluxo, sem transformar a estrutura de arquivos do código em arquitetura do artigo.

5.1. Do Contrato ao Plano de Execução

O analisador aceita documentos OpenAPI em YAML ou JSON e os converte em um modelo interno de operações, parâmetros, corpos, respostas e requisitos de segurança. O subconjunto implementado reconhece GET, POST, PUT, PATCH e DELETE; parâmetros de caminho, consulta e cabeçalho; corpos e respostas JSON; referências locais; e composição limitada por `allOf`. Restrições como obrigatoriedade, `readOnly`, enumeração, formato, padrão, tamanhos e limites numéricos são preservadas quando suportadas.

O gerador produz a entrada-base e o motor de mutações percorre sua estrutura para aplicar alterações tipadas. O construtor do plano resolve caminhos, corpos, cabeçalhos, autenticação e estratégia aplicada. Cada entrada recebe um `case_id` e registra se representa caso-base, mutação ou teste de autenticação, além dos metadados das mutações. O plano completo é persistido antes das chamadas à API.

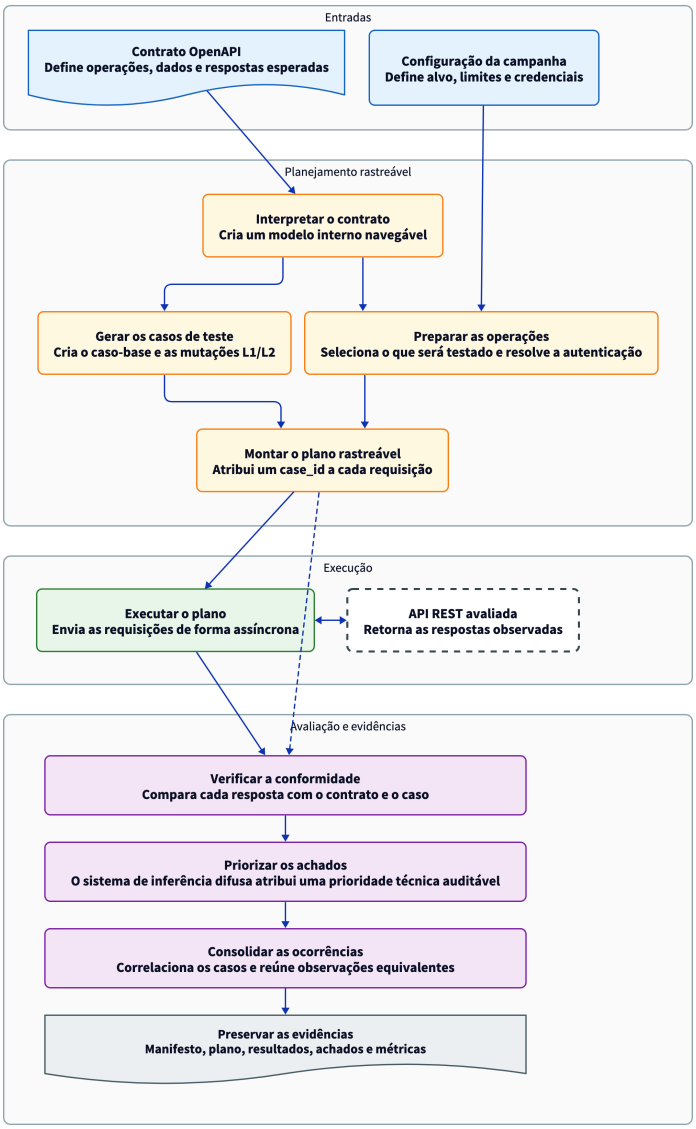


Figura 2. Arquitetura interna do OpenAPI Contract Fuzzer. Contrato e parâmetros são transformados em um plano rastreável; as respostas da API são confrontadas com o comportamento documentado, priorizadas pelo sistema *fuzzy* e consolidadas em artefatos auditáveis.

5.2. Execução, Análise e Rastreabilidade

A execução utiliza cliente HTTP assíncrono, semáforo de concorrência, atraso e tempo limite configuráveis. Para cada caso são registrados URL, status, tempo, tamanho, corpo e erro de transporte. Os executores experimentais cuidam da inicialização, da reinicialização e da proveniência específica das aplicações sem alterar o núcleo da ferramenta.

O analisador confronta status e esquemas observados com o contrato. Entre as classificações estão falhas de transporte, erros 5xx, status não documentados, incompatibilidades de esquema e aceitação de entradas que deveriam ser rejeitadas. Uma rejeição 4xx documentada e compatível é preservada como comportamento esperado e separada dos candidatos. Uma

divergência contratual continua relevante mesmo quando a API rejeita a entrada com segurança, pois pode afetar consumidores; por outro lado, uma observação reportada não equivale automaticamente a causa raiz, defeito confirmado ou vulnerabilidade.

O pós-processamento correlaciona plano, resultado e achado por `case_id`, constrói assinaturas, seleciona representantes e calcula métricas. Cada diretório de campanha reúne `run_manifest.json`, plano de execução, resultados brutos, achados estruturados, conjunto deduplicado e relatórios em Markdown. O manifesto preserva configuração, revisão e estado do código, resumos criptográficos do contrato e da árvore-fonte, ambiente, horários, contagens e estado da campanha. Cabeçalhos e valores sensíveis são redigidos antes da persistência pública.

Figura 3. Níveis de observação e rastreabilidade dos achados. O `case_id` relaciona plano, resposta e classificação; a deduplicação produz achados operacionais únicos, e o agrupamento em famílias ocorre em etapa analítica posterior.

Esse conjunto permite rastrear os números do artigo até casos concretos e distinguir falha da aplicação, falha ambiental e campanha incompleta. Uma amostra pública gerada pelo fluxo contém uma execução L2 de cada alvo e demonstra o formato dos artefatos, mas não define o modelo de coleta final.

5.3. Priorização *Fuzzy*

Depois da classificação contratual, um sistema Mamdani combina sinais observáveis para produzir um escore contínuo e as faixas `LOW`, `MEDIUM` e `HIGH`. O FIS não decide se existe divergência; ele organiza a inspeção dos achados já reportados. Entradas, pertinências, regras ativadas e eventual regra alternativa são preservados nos artefatos para tornar a decisão auditável.

O construto é prioridade técnica da anomalia, abrangendo *contract drift*, compatibilidade funcional, robustez e controle de acesso. O escore não estima impacto real, explorabilidade nem gravidade de segurança. A evolução do modelo e os sinais utilizados em sua revisão são apresentados uma única vez na Seção 6.2, junto das evidências que motivaram a mudança.

A cobertura de OpenAPI permanece parcial. Outros tipos de mídia, `oneOf`, `anyOf`, referências externas e parte das regras de serialização não são suportados. A ferramenta também não resolve dependências gerais entre recursos ou sequências com estado; identificadores dinâmicos podem apontar para recursos inexistentes. Esses limites restringem as operações exercitáveis e acompanham a interpretação dos resultados.

6. Resultados e Discussão

Os resultados disponíveis descrevem uma trajetória experimental: os pilotos caracterizaram o funcionamento do fluxo e expuseram limitações do mecanismo de priorização; a auditoria orientou uma revisão; e o modelo congelado foi aplicado posteriormente a uma API que não participou de seu desenvolvimento. Essa sequência permite apresentar evidências parciais sem confundir corpus de desenvolvimento, confirmação externa e uma futura coleta final.

6.1. Caracterização dos Pilotos e dos Níveis de Mutação

As campanhas de Nível 2 contra VAmPI, Spring Petclinic REST, crAPI e RealWorld executaram 958 de 958 requisições. Desse total, 284 eram casos L2 com duas mutações; os demais preservaram

casos-base e mutações isoladas. A deduplicação produziu 265 assinaturas estritas, reunidas em 161 famílias comportamentais. A referência cega distribuiu essas famílias em 99 `LOW`, 59 `MEDIUM` e 3 `HIGH`.

O Nível 0 forneceu a observação nominal das operações; o Nível 1 isolou o efeito dos operadores; e o Nível 2 ampliou a variedade de manifestações, especialmente na crAPI. Entretanto, as combinações não produziram uma nova família `HIGH` exclusiva. O resultado não torna L2 irrelevante: interações entre entradas, *contract drift*, respostas incompatíveis e variações de robustez também fazem parte do objeto, mesmo quando não indicam vulnerabilidade grave.

6.2. Auditoria e Revisão do Modelo de Priorização

A configuração inicial do FIS combinava desvio de status HTTP, tempo de resposta e tamanho da resposta. A avaliação cega mostrou que o status dominava as decisões: neutralizar tempo ou tamanho não alterou a faixa de nenhuma das 161 famílias. O modelo e a linha de base baseada somente em status obtiveram a mesma acurácia, 0,298, o mesmo macro-F1, 0,254, e o mesmo erro ordinal médio, 0,702.

A matriz de confusão revelou 67 famílias `LOW` elevadas a `MEDIUM` e 43 famílias `MEDIUM` elevadas a `HIGH`. O mecanismo superpriorizava rejeições seguras, 4xx não documentados e falhas 5xx localizadas. Uma rejeição não documentada pode representar *contract drift*, mas não deve receber automaticamente a mesma prioridade de uma operação válida indisponível, de uma aceitação indevida ou de uma falha recorrente.

Esse diagnóstico levou à revisão do construto e dos antecedentes. O modelo revisado utiliza contradição comportamental, força da evidência e exposição de controle de acesso. A contradição representa a incompatibilidade entre o caso, o status, o esquema e o comportamento observado; a evidência distingue manifestações isoladas de determinadas recorrências; e a exposição de acesso captura operações protegidas que aceitam credencial ausente ou inválida. Seis regras Mamdani agregam esses sinais e produzem o escore contínuo.

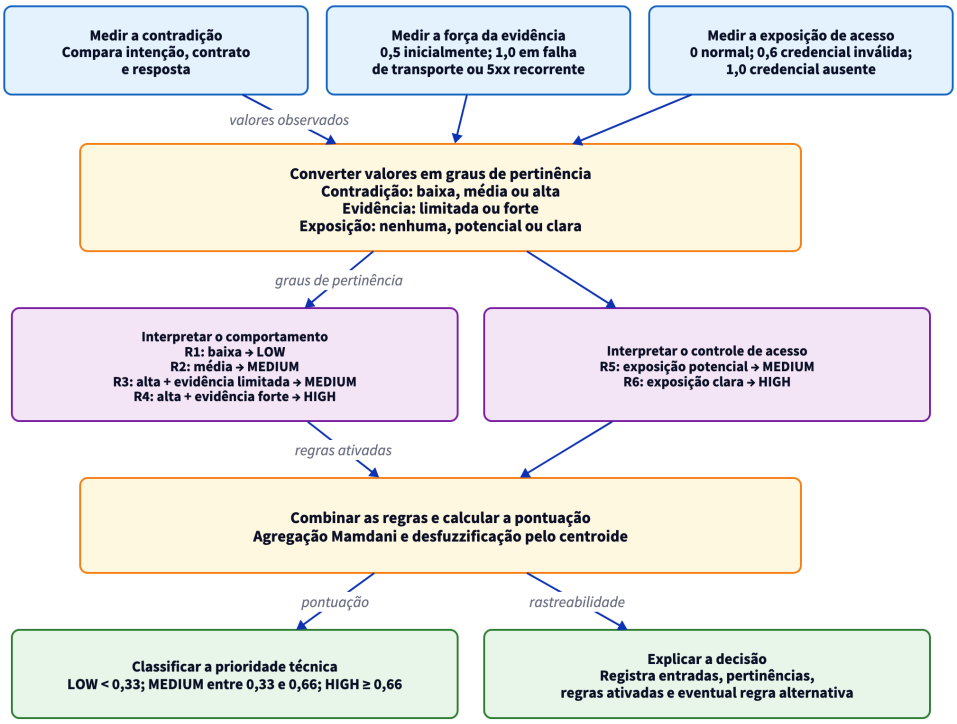


Figura 4. Fluxo auditável do sistema Mamdani de priorização. Sinais do contrato, da resposta e da autenticação são fuzzificados e combinados por seis regras. A saída representa prioridade técnica, não confirmação de vulnerabilidade.

No conjunto de desenvolvimento, o modelo revisado reproduziu as 161 faixas da rubrica, com macro-F1 1,000 e erro ordinal zero. Esse resultado demonstra que a implementação expressa a rubrica formulada, mas não constitui evidência de generalização, pois a mesma amostra foi usada para diagnosticar e revisar o mecanismo. Por isso, regras e parâmetros foram congelados antes da avaliação externa.

6.3. Avaliação Externa no Meilisearch

O modelo congelado foi aplicado ao Meilisearch v1.45.1 em cinco campanhas, cada uma com 85 operações selecionadas e 477 requisições concluídas. As assinaturas estritas apresentaram interseção 216 e união 276. A variação veio dos campos `requestUid` e `processingTimeMs` em `POST /multi-search`. Após normalização restrita a esses dois campos, cada repetição apresentou as mesmas 222 assinaturas semânticas.

As assinaturas foram agrupadas em 134 famílias. A referência final classificou 122 como sustentadas e 12 como indeterminadas. Entre as sustentadas, 116 receberam `LOW`, 6 receberam `MEDIUM` e nenhuma recebeu `HIGH`.

O modelo revisado concordou com a referência em 120 das 122 famílias sustentadas; a linha de base baseada somente em status acertou 4. A Tabela 2 apresenta as métricas nas classes observadas.

Tabela 2. Comparação confirmatória nas 122 famílias sustentadas

Métrica	Modelo revisado	Somente status
Acurácia	0,9836	0,0328
Macro-F1 (classes observadas)	0,8957	0,0317
Acurácia balanceada	0,8333	0,3333
Erro ordinal médio	0,0164	0,9672
Spearman	0,9088	-0,6981

Como 84 das 122 famílias sustentadas eram controles de credencial inválida, foi realizada uma análise sem esses casos. O modelo revisado permaneceu correto em 36 de 38 famílias, contra 4 de 38 da linha de base. Os dois desacordos foram preservados: um caso-base de `PATCH /experimental-features` com status 415 não documentado e uma mutação válida de `POST /export` que retornou 200 quando o contrato documentava 202. O FIS não foi reajustado com esses casos.

6.4. Discussão e Limitações

O principal resultado do corpus de desenvolvimento foi explicar por que o modelo inicial não acrescentava separação categórica ao status. A revisão substituiu sinais fáceis de medir, mas pouco discriminativos, por fatos mais próximos da prioridade técnica. A avaliação externa indica que essa operacionalização permaneceu coerente no Meilisearch sem retroalimentar o modelo congelado.

Os resultados não permitem avaliar a faixa `HIGH` no *holdout*, estimar gravidade real de segurança ou extrapolar para APIs em geral. A concordância de prioridade entre os agentes foi baixa na confirmação, e a amostra apresentou forte presença de controles de credencial inválida. A referência deve ser entendida como operacional e auditável, não como verdade universal.

A contribuição observada é um processo rastreável para gerar, detectar, deduplicar e priorizar anomalias técnicas, incluindo *contract drift*, robustez e controle de acesso. A coleta final ampliará a avaliação de execução, deduplicação e estabilidade às cinco APIs, sem repetir a rotulagem cega. A comparação de prioridade aqui apresentada permanece restrita aos conjuntos históricos; Schemathesis integra somente a comparação bibliográfica.

7. Conclusão

Este esboço do artigo registra o estado atual do projeto, não o fechamento da avaliação. Foi implementado um fluxo que interpreta um subconjunto de OpenAPI, gera casos-base e mutações, executa testes *black-box*, identifica divergências, deduplica observações, atribui prioridade técnica e preserva artefatos rastreáveis por campanha.

Os pilotos mostraram que os níveis de mutação exercitam perspectivas complementares: comportamento nominal, alterações isoladas e interações. A auditoria do mecanismo de priorização revelou que sua configuração inicial era dominada pelo status HTTP. A revisão orientada por fatos comportamentais foi congelada e apresentou concordância substancialmente maior que a linha de base no *holdout* Meilisearch.

A contribuição não é a determinação automática de vulnerabilidades ou impacto real, mas uma forma auditável de priorizar *contract drift*, incompatibilidades funcionais, problemas de

robustez e indícios de controle de acesso. As conclusões são limitadas pelo número de aplicações, pelo OpenAPI como oráculo parcial, pela referência assistida por agentes, pelo desbalanceamento das classes e pela ausência de casos `HIGH` no *holdout*.

A próxima etapa prevê cinco campanhas por aplicação para avaliar execução, deduplicação e estabilidade, mantendo a prioridade como resultado descritivo nessa nova coleta. A avaliação histórica do FIS será preservada, sem nova rotulagem cega ou recalibração. Schemathesis permanece nos trabalhos relacionados, sem comparação experimental.

8. Referências

Alakeel, A. M. (2014). Using fuzzy logic in test case prioritization for regression testing programs with assertions. *The Scientific World Journal*, 2014.

Atlidakis, V., Godefroid, P., e Polishchuk, M. (2019). Restler: Stateful rest api fuzzing. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, pages 748–758.

Corradini, D., Zampieri, A., Pasqua, M., e Ceccato, M. (2021). Empirical comparison of black-box test case generation tools for restful apis. In *2021 IEEE 21st International Working Conference on Source Code Analysis and Manipulation (SCAM)*, pages 226–236.

Corradini, D., Zampieri, A., Pasqua, M., e Ceccato, M. (2022). Resttestgen: An extensible framework for automated black-box testing of restful apis. In *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 504–508.

Elbaum, S., Malishevsky, A. G., e Rothermel, G. (2002). Test case prioritization: A family of empirical studies. *IEEE Transactions on Software Engineering*, 28(2):159–182.

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine.

Fielding, R. T., Nottingham, M., e Reschke, J. (2022). Http semantics. RFC 9110.

Golmohammadi, A., Zhang, M., e Arcuri, A. (2023). Testing restful apis: A survey. *ACM Transactions on Software Engineering and Methodology*, 33(1).

Hatfield-Dodds, Z. e Dygalo, D. (2021). Deriving semantics-aware fuzzers from web api schemas. Apresentado como pôster na ICSE 2022.

JSON Schema (2022). Json schema 2020-12.

Mamdani, E. H. e Assilian, S. (1975). An experiment in linguistic synthesis with a fuzzy logic controller. *International Journal of Man-Machine Studies*, 7(1):1–13.

Martin-Lopez, A., Segura, S., e Ruiz-Cortes, A. (2020). Restest: Black-box constraint-based testing of restful web apis. In *Service-Oriented Computing*, volume 12571 of *Lecture Notes in Computer Science*, pages 459–475.

Meyer, B. (1992). Applying "design by contract". *IEEE Computer*, 25(10):40–51.

OpenAPI Initiative (2021). Openapi specification 3.1.0.

Zadeh, L. A. (1965). Fuzzy sets. *Information and Control*, 8(3):338–353.

Zhang, M. e Arcuri, A. (2023). Open problems in fuzzing restful apis: A comparison of tools. *ACM Trans. Softw. Eng. Methodol.*, 32(6).